Table of Contents

..... 1
========== SETTINGS ========== 2
========== LOAD RESULTS ========== 3
========== PREPARE ANGLES ========== 4
========== PREDICTION LABELS ========== 4
========== GESTURE TARGETS ========== 5
========== STATE ========== 6
========== FIGURE ========== 7
========== INFO PANEL ========== 8
========== MODE ========== 9
========== SPEED ========== 10
========== CAMERA ========== 11
========== SLIDER ========== 12
========== TIMER ========== 13
========== INITIAL DISPLAY ========== 14
========== 14
========== 16
========== 18
========== 19
========== 23
========== 27
========== 29
========== 30


```
function visualizeController()

% VISUALIZECONTROLLER
% MATLAB R2025b prosthetic-hand visualizer with an anatomically organised
% 3-D hand (palm, wrist, 4 fingers, side-mounted thumb).
%
% INPUT:
%   ControllerResults.mat  (searched in this file's folder, then pwd)
%     actualAngleHistory / actualAngles       [N x 5]  radians (degrees auto-
detected)
%     desiredAngleHistory / desiredAngles     [N x 5]
%     predictedLabel / predictedLabels / currentGesture / gesture   [N x 1]
%     dt
%   Random_forests/PredictionResults_RF.mat (optional fallback for labels;
%     also accepted next to this file or in pwd)
%
% Finger order everywhere:  [Thumb Index Middle Ring Little]
%
% HAND COORDINATE SYSTEM (units ~ cm)
%   X : across the palm. +X = thumb side (little finger at -X)
%   Y : palm depth.      -Y = palm side (fingers FLEX toward -Y)
%   Z : wrist -> fingers. Palm centre = [0 0 0], wrist toward -Z.
%   Default camera looks at the palm face (like the reference photo).
%
% TIMER:  Period is FIXED at 0.05 s. The Speed menu only changes how many
```

```
%   frames are advanced per tick; timerObj.Period is never modified.
%
% NOTE: gesture->pose is a visualization mapping; the finger angle drives a
%   3-joint chain using JOINT_GAIN_* below (see SETTINGS).

clc;
close all;
```

========== SETTINGS ==========

```
LABEL_BASE   = 0;        % DB1 convention: 0...52
DEFAULT_MODE = 1;        % 1 Gesture Target, 2 Actual PID, 3 Blend
BLEND_ALPHA  = 0.50;
TIMER_PERIOD = 0.05;     % FIXED; do not make this depend on speed
FINGER_NAMES = {'Thumb','Index','Middle','Ring','Little'};
SPEED_VALUES = [0.25 0.5 1 2 4 8 16 32];
SPEED_LABELS = {'0.25x','0.5x','1x','2x','4x','8x','16x','32x'};
PRINT_AUDIT  = true;     % print a frame audit table to the command window

% --- view ------------------------------------------------------------
VIEW_AZ = -28;  VIEW_EL = 18;
AX_X = [-9 13];  AX_Y = [-8 8];  AX_Z = [-12 16.5];

% --- palm / wrist ----------------------------------------------------
PALM_HALF_W = 4.4;   % half width across knuckles (X)
PALM_HALF_T = 1.7;   % half thickness (Y)
PALM_HALF_L = 4.8;   % half length (Z)

% --- four fingers, geometry index: 1 Index, 2 Middle, 3 Ring, 4 Little
FINGER_BASE_X   = [ 2.30  0.55 -1.25 -3.05];
FINGER_BASE_Z   = [ 3.90  4.20  3.95  3.40];
FINGER_LEN      = [ 7.90  8.80  8.10  6.60];   % Middle > Ring ~ Index >
Little
FINGER_FAN_DEG  = [ 5     0    -5    -11  ];   % outward fan (+ = toward
thumb)
FINGER_WIDTH    = [ 1.00  1.00  0.95  0.85];
FINGER_SEG_FRAC = [0.47 0.30 0.23];            % proximal / middle / distal

% One commanded finger angle theta is shared by MCP, PIP, DIP:
%   joint_i = min(GAIN_i*theta, MAX_i); cumulative sums give the segment
%   directions (MCP moves everything, PIP moves middle+distal, DIP distal).
JOINT_GAIN_FINGER    = [1.10 1.30 0.80];
JOINT_MAX_FINGER_DEG = [95 110 80];

% --- thumb (separate kinematics) -------------------------------------
THUMB_BASE      = [3.1 -0.55 -3.3];   % CMC joint, inside the thenar region
THUMB_LEN       = [4.6 3.5 3.0];      % metacarpal / proximal / distal
THUMB_REST_DEG  = [36 46 54];         % rest direction from +Z toward +X
JOINT_GAIN_THUMB    = [0.45 0.75 0.70];
JOINT_MAX_THUMB_DEG = [40 70 80];
```

```
% --- appearance ----------------------------------------------------------
COL_PALM  = [0.62 0.66 0.72];
COL_WRIST = [0.45 0.50 0.56];
COL_CUFF  = [0.20 0.25 0.31];
COL_BONE  = [0.74 0.78 0.83];
COL_JOINT = [0.20 0.25 0.31];
SURF_OPTS = {'EdgeColor','none','FaceLighting','gouraud', ...
             'AmbientStrength',0.45,'DiffuseStrength',0.70, ...
             'SpecularStrength',0.12};
```

========== LOAD RESULTS ==========

```
currentFolder = fileparts(mfilename('fullpath'));
if isempty(currentFolder)
    currentFolder = pwd;
end

resultFile = locateFile('ControllerResults.mat','');

if isempty(resultFile)
    error(['ControllerResults.mat was not found.' newline newline ...
           'Searched:' newline '  %s' newline '  %s' newline newline ...
           'Run runController.m first.'], ...
           fullfile(currentFolder,'ControllerResults.mat'), ...
           fullfile(pwd,'ControllerResults.mat'));
end

S = load(resultFile);

actualHistory = getVariableRecursive(S, ...
    {'actualAngleHistory','actualAngles'});

desiredHistory = getVariableRecursive(S, ...
    {'desiredAngleHistory','desiredAngles'});

gestureVector = getVariableRecursive(S, ...
    {'predictedLabel','predictedLabels','currentGesture','gesture'});

dt = getVariableRecursive(S,{'dt'});

if isempty(dt) || ~isnumeric(dt) || ~isscalar(dt) || ...
        ~isfinite(dt) || dt <= 0
    dt = 0.01;
    warning('dt missing/invalid in ControllerResults.mat; using %.3f s.',dt);
else
    dt = double(dt);
end
```

========== PREPARE ANGLES ==========

```
actualHistory  = prepareAngleHistory(actualHistory);
desiredHistory = prepareAngleHistory(desiredHistory);

if isempty(actualHistory) && isempty(desiredHistory)
    error(['No usable finger-angle history was found.' newline ...
           'Expected actualAngleHistory/actualAngles or ' ...
           'desiredAngleHistory/desiredAngles (N x 5).']);
end

if isempty(actualHistory)
    warning('No actual PID angles found; Actual PID mode shows desired
angles.');
    actualHistory = desiredHistory;
end

if isempty(desiredHistory)
    warning('No desired angles found; using actual angles as desired.');
    desiredHistory = actualHistory;
end

angleFrames = max(size(actualHistory,1),size(desiredHistory,1));
```

========== PREDICTION LABELS ==========

```
gestureVector = prepareGestureVector(gestureVector);

% If ControllerResults.mat has no usable labels, use the Random Forest file.
if numel(gestureVector) <= 1
    predictionFile = locateFile('PredictionResults_RF.mat','Random_forests');

    if isempty(predictionFile)
        warning(['No predicted labels in ControllerResults.mat and ' ...
                 'PredictionResults_RF.mat was not found. Gesture will ' ...
                 'be shown as Unknown / N/A.']);
    else
        rfData   = load(predictionFile);
        rfLabels = prepareGestureVector( ...
            getVariableRecursive(rfData,
{'predictedLabel','predictedLabels'}));

        if numel(rfLabels) > 1
            gestureVector = rfLabels;
            fprintf('Using Random Forest predictions: %d labels (%s)\n', ...
                    numel(gestureVector),predictionFile);
        else
            warning('PredictionResults_RF.mat has no usable
```

```
predictedLabel.');
        end
    end
end

% Align labels to the angle frames. runController.m repeats every RF label
% by an integer factor, so a shorter label vector is repeated, not stretched.
nLabels = numel(gestureVector);
if nLabels > 1 && nLabels < angleFrames
    if mod(angleFrames,nLabels) == 0
        gestureVector = repelem(gestureVector,angleFrames/nLabels);
        fprintf('Labels repeated x%d to match %d angle frames.\n', ...
                angleFrames/nLabels,angleFrames);
    else
        warning(['Label count (%d) does not divide frame count (%d); ' ...
                 'last label is held for the remaining frames.'], ...
                 nLabels,angleFrames);
    end
end

hasLabels = any(isfinite(gestureVector));

N = max(angleFrames,numel(gestureVector));

if N < 1
    error('No controller frames are available.');
end

actualHistory  = resizeHistory(actualHistory,N);
desiredHistory = resizeHistory(desiredHistory,N);
gestureVector  = resizeVector(gestureVector,N);

actualHistory  = convertToRadiansIfNeeded(actualHistory);
desiredHistory = convertToRadiansIfNeeded(desiredHistory);

actualHistory(~isfinite(actualHistory))   = 0;
desiredHistory(~isfinite(desiredHistory)) = 0;
```

========== GESTURE TARGETS ==========

```
targetHistory = zeros(N,5);

for k = 1:N
    if isfinite(gestureVector(k))
        targetHistory(k,:) = gestureToPose(gestureVector(k),LABEL_BASE);
    else
        % No label for this frame: fall back to the controller's own
        % desired angles (never invented values).
        targetHistory(k,:) = desiredHistory(k,:);
    end
end
```

```
targetHistory(~isfinite(targetHistory)) = 0;

fprintf('\n=============================================\n');
fprintf('3-D PROSTHETIC HAND CONTROLLER\n');
fprintf('=============================================\n');
fprintf('Frames       : %d\n',N);
fprintf('dt           : %.4f sec\n',dt);
fprintf('Timer period : %.3f sec (fixed)\n',TIMER_PERIOD);
fprintf('Labels       : %s\n',ternary(hasLabels,'loaded','MISSING (Unknown /
N/A)'));
fprintf('=============================================\n');

if PRINT_AUDIT
    printFrameAudit();
end
```

```
=============================================
3-D PROSTHETIC HAND CONTROLLER
=============================================
Frames       : 40596
dt           : 0.0500 sec
Timer period : 0.050 sec (fixed)
Labels       : loaded
=============================================
FRAME AUDIT (raw loaded data, degrees)
  frame  label | target T  I  M  R  L        | actual T  I  M  R  L
      1      0 |   0.0  0.0  0.0  0.0  0.0 |   0.0  0.0  0.0  0.0  0.0
      2      0 |   0.0  0.0  0.0  0.0  0.0 |   0.0  0.0  0.0  0.0  0.0
      3      0 |   0.0  0.0  0.0  0.0  0.0 |   0.0  0.0  0.0  0.0  0.0
    101     19 |  45.0  5.0 70.0 70.0 70.0 |  16.2  0.9 24.5 83.6 26.9
    102     19 |  45.0  5.0 70.0 70.0 70.0 |  26.1  1.6 39.5 97.5 41.9
  40596      0 |   0.0  0.0  0.0  0.0  0.0 |  -0.0  5.5  2.2  1.9 -0.8
```

========== STATE ==========

```
currentFrame = 1;
isPlaying    = false;
playSpeed    = 1;
frameAccum   = 0;
poseMode     = DEFAULT_MODE;
timerObj     = [];
staticGroup  = [];
dynGroup     = [];
```

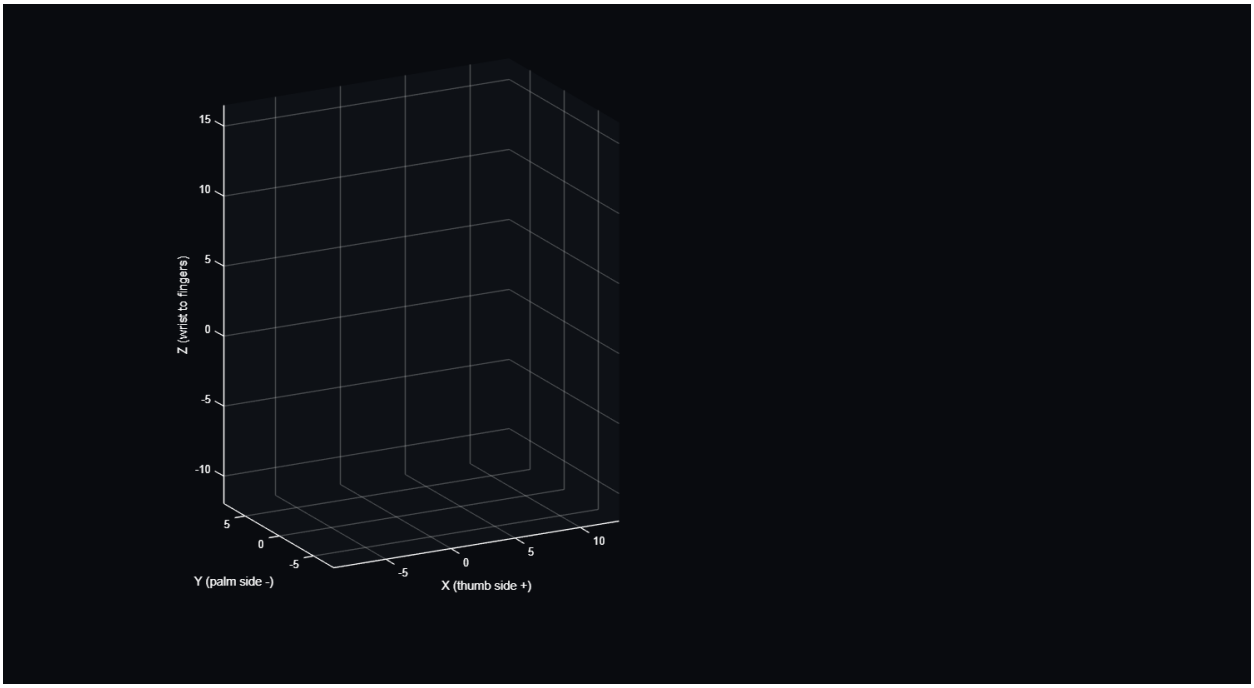
========== FIGURE ==========

```
fig = figure( ...
    'Name','3-D Prosthetic Hand Controller', ...
    'NumberTitle','off', ...
    'Color',[0.035 0.045 0.060], ...
    'Position',[50 40 1500 850], ...
    'CloseRequestFcn',@closeFigure, ...
    'DeleteFcn',@cleanupTimer);

ax = axes( ...
    'Parent',fig, ...
    'Units','normalized', ...
    'Position',[0.035 0.17 0.60 0.75], ...
    'Color',[0.055 0.065 0.085], ...
    'XColor','w','YColor','w','ZColor','w', ...
    'FontSize',10);

hold(ax,'on');
grid(ax,'on');
axis(ax,'equal');
xlim(ax,AX_X);
ylim(ax,AX_Y);
zlim(ax,AX_Z);
view(ax,VIEW_AZ,VIEW_EL);

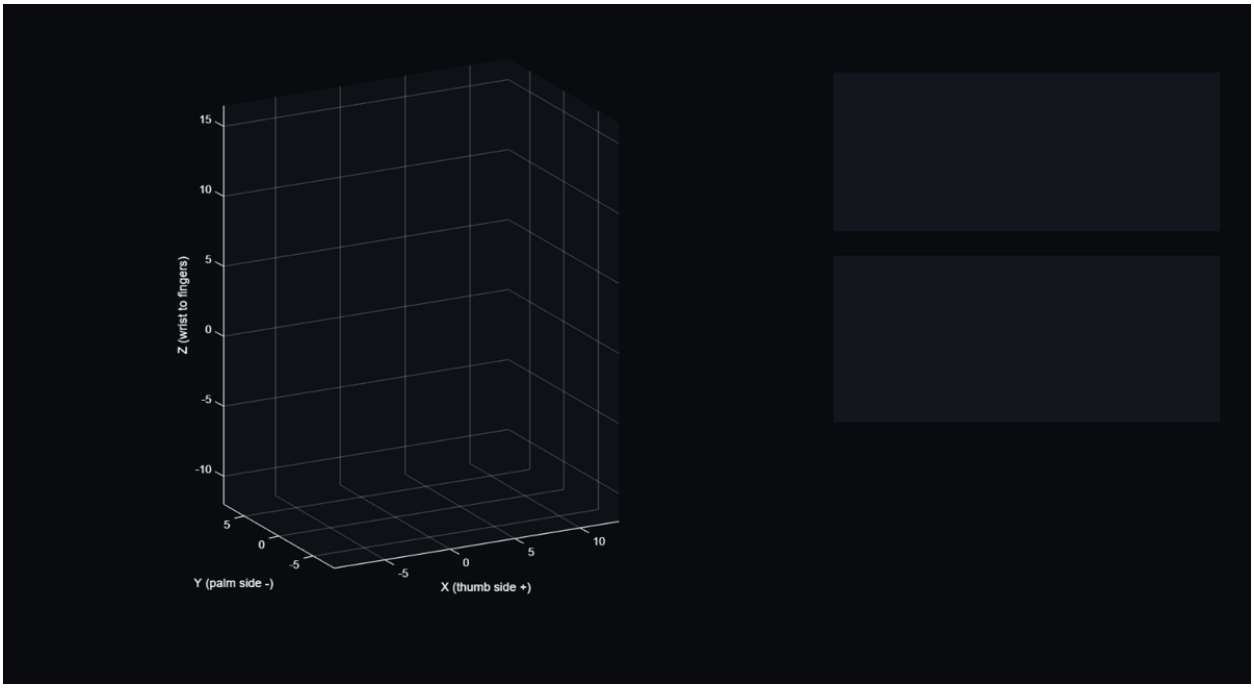
xlabel(ax,'X (thumb side +)','Color','w');
ylabel(ax,'Y (palm side -)','Color','w');
zlabel(ax,'Z (wrist to fingers)','Color','w');
```



========== INFO PANEL ==========

```
infoText = uicontrol(fig, ...
    'Style','text', ...
    'String','', ...
    'Units','normalized', ...
    'Position',[0.665 0.665 0.31 0.235], ...
    'BackgroundColor',[0.075 0.085 0.11], ...
    'ForegroundColor','w', ...
    'HorizontalAlignment','left', ...
    'FontName','monospaced', ...
    'FontSize',11);

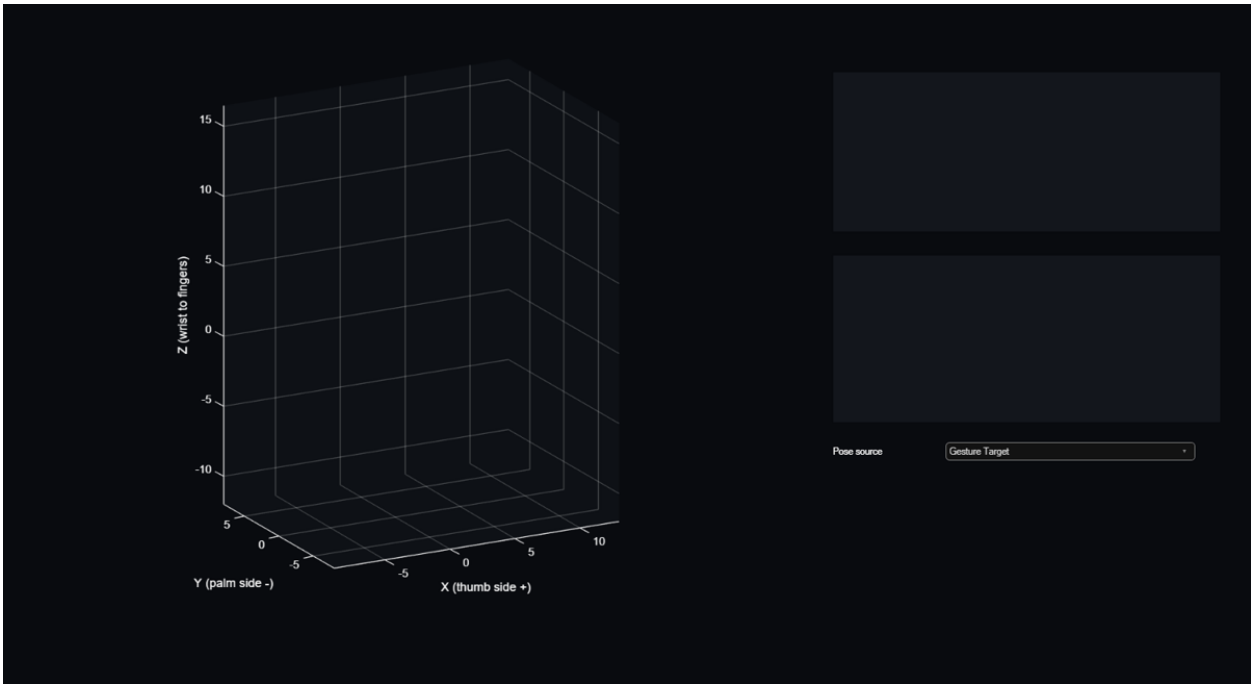
angleText = uicontrol(fig, ...
    'Style','text', ...
    'String','', ...
    'Units','normalized', ...
    'Position',[0.665 0.385 0.31 0.245], ...
    'BackgroundColor',[0.075 0.085 0.11], ...
    'ForegroundColor','w', ...
    'HorizontalAlignment','left', ...
    'FontName','monospaced', ...
    'FontSize',10);
```



========== MODE ==========

```
uicontrol(fig,'Style','text', ...
    'String','Pose source', ...
    'Units','normalized', ...
    'Position',[0.665 0.315 0.09 0.035], ...
    'BackgroundColor',[0.035 0.045 0.060], ...
    'ForegroundColor','w', ...
    'HorizontalAlignment','left');

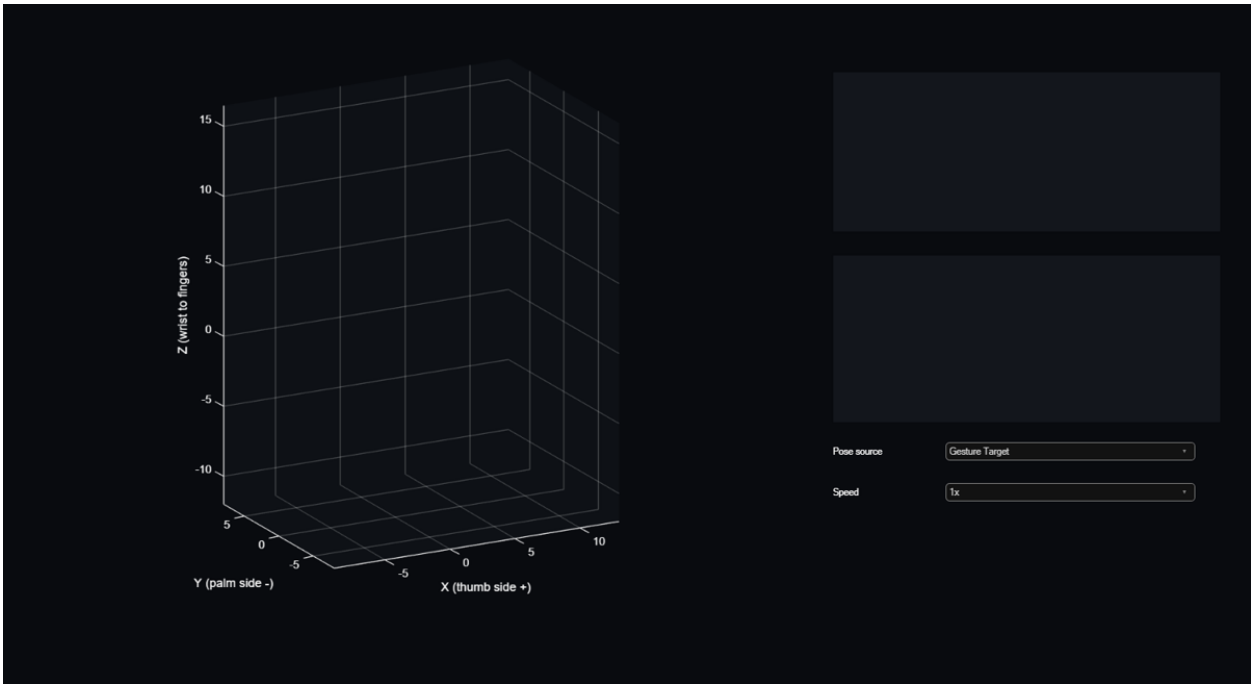
modeMenu = uicontrol(fig, ...
    'Style','popupmenu', ...
    'String',{'Gesture Target','Actual PID','Blend'}, ...
    'Value',poseMode, ...
    'Units','normalized', ...
    'Position',[0.755 0.315 0.20 0.04], ...
    'Callback',@changeMode); %#ok<NASGU>
```



========== SPEED ==========

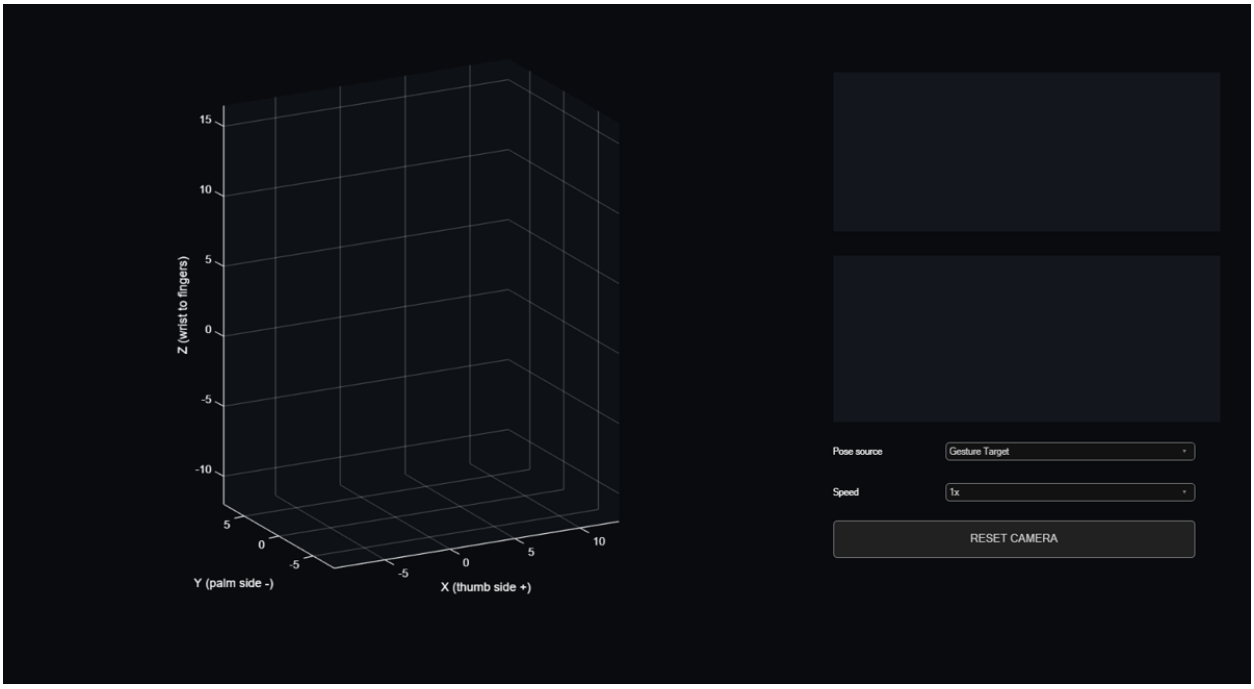
```
uicontrol(fig,'Style','text', ...
    'String','Speed', ...
    'Units','normalized', ...
    'Position',[0.665 0.255 0.09 0.035], ...
    'BackgroundColor',[0.035 0.045 0.060], ...
    'ForegroundColor','w', ...
    'HorizontalAlignment','left');

speedMenu = uicontrol(fig, ...
    'Style','popupmenu', ...
    'String',SPEED_LABELS, ...
    'Value',3, ...
    'Units','normalized', ...
    'Position',[0.755 0.255 0.20 0.04], ...
    'Callback',@changeSpeed); %#ok<NASGU>
```



========== CAMERA ==========

```
uicontrol(fig, ...
    'Style','pushbutton', ...
    'String','RESET CAMERA', ...
    'Units','normalized', ...
    'Position',[0.665 0.185 0.29 0.055], ...
    'FontSize',10, ...
    'Callback',@resetCamera);
```



========== SLIDER ==========

```
slider = uicontrol(fig, ...
    'Style','slider', ...
    'Units','normalized', ...
    'Position',[0.075 0.095 0.57 0.035], ...
    'Min',1, ...
    'Max',max(2,N), ...
    'Value',1, ...
    'SliderStep',[1/max(1,N-1),min(20/max(1,N-1),1)], ...
    'Callback',@sliderCallback);

frameText = uicontrol(fig, ...
    'Style','text', ...
    'String','Frame: 1', ...
    'Units','normalized', ...
    'Position',[0.075 0.045 0.17 0.04], ...
    'BackgroundColor',[0.035 0.045 0.060], ...
    'ForegroundColor','w', ...
    'FontSize',11, ...
    'HorizontalAlignment','left');

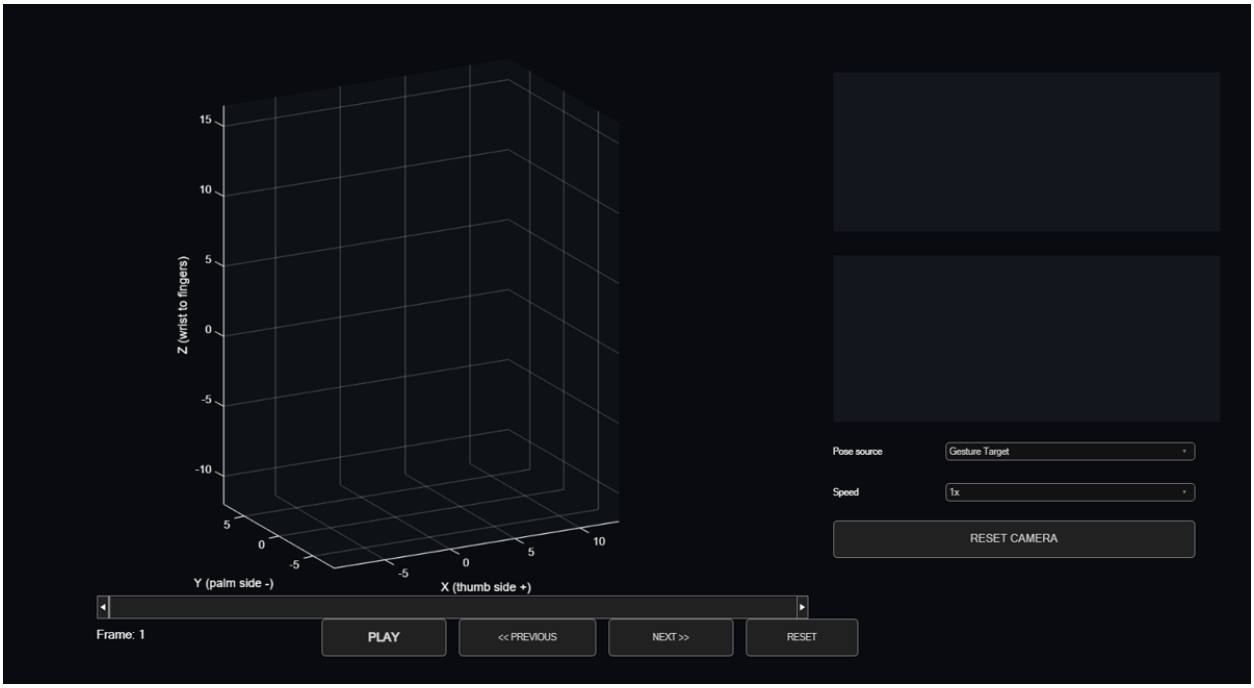
playButton = uicontrol(fig, ...
    'Style','pushbutton', ...
    'String','PLAY', ...
    'Units','normalized', ...
    'Position',[0.255 0.04 0.10 0.055], ...
    'FontSize',11, ...
```

```
    'FontWeight','bold', ...
    'Callback',@playPause);

uicontrol(fig, ...
    'Style','pushbutton', ...
    'String','<< PREVIOUS', ...
    'Units','normalized', ...
    'Position',[0.365 0.04 0.11 0.055], ...
    'Callback',@previousFrame);

uicontrol(fig, ...
    'Style','pushbutton', ...
    'String','NEXT >>', ...
    'Units','normalized', ...
    'Position',[0.485 0.04 0.10 0.055], ...
    'Callback',@nextFrame);

uicontrol(fig, ...
    'Style','pushbutton', ...
    'String','RESET', ...
    'Units','normalized', ...
    'Position',[0.595 0.04 0.09 0.055], ...
    'Callback',@resetFrame);
```



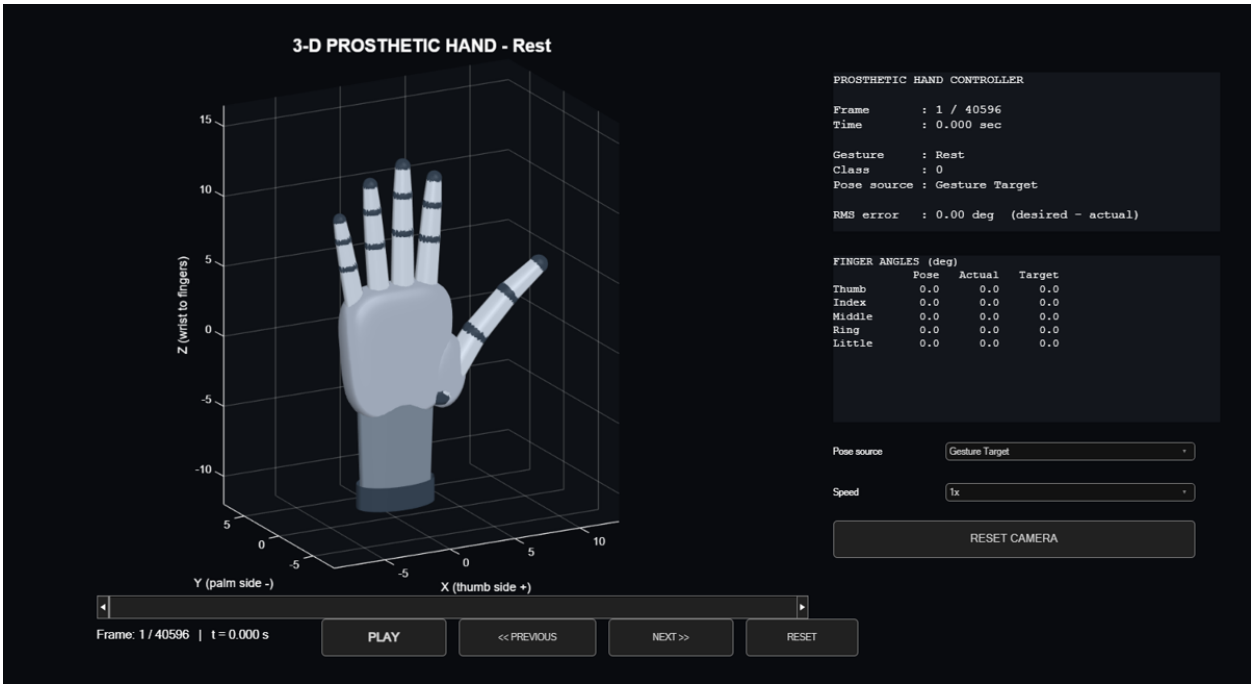
========== TIMER ==========

Timer period is constant. Speed is handled by frame increment in timerCallback(); there is intentionally no timerObj.Period assignment anywhere else in this file.

```
timerObj = timer( ...
    'ExecutionMode','fixedSpacing', ...
    'Period',TIMER_PERIOD, ...
    'BusyMode','drop', ...
    'TimerFcn',@timerCallback);
```

========== INITIAL DISPLAY ==========

```
updateDisplay();
drawnow;
start(timerObj);
```



==========

==========

CALLBACKS ==========

```
    function sliderCallback(src,~)
        currentFrame = round(get(src,'Value'));
        currentFrame = max(1,min(N,currentFrame));
        isPlaying = false;
        frameAccum = 0;
        set(playButton,'String','PLAY');
        updateDisplay();
    end

    function playPause(~,~)
```

```
        isPlaying = ~isPlaying;

        if isPlaying
            if currentFrame >= N       % restart from the beginning
                currentFrame = 1;
                set(slider,'Value',1);
            end
            frameAccum = 0;
            set(playButton,'String','PAUSE');
        else
            set(playButton,'String','PLAY');
        end
    end

    function timerCallback(~,~)
        if ~isvalidFigure(fig)
            cleanupTimer();
            return;
        end

        if ~isPlaying
            return;
        end

        % Frames advanced per tick = speed * period / dt (fractional
        % remainder is accumulated so 0.25x / 0.5x really are slower).
        % timerObj.Period is NOT touched.
        frameAccum = frameAccum + playSpeed * TIMER_PERIOD / dt;
        frameStep  = floor(frameAccum + 1e-9);

        if frameStep < 1
            return;
        end

        frameAccum = frameAccum - frameStep;
        currentFrame = currentFrame + frameStep;

        if currentFrame >= N
            currentFrame = N;
            isPlaying = false;
            set(playButton,'String','PLAY');
        end

        set(slider,'Value',currentFrame);
        updateDisplay();
    end

    function nextFrame(~,~)
        isPlaying = false;
        set(playButton,'String','PLAY');

        currentFrame = min(N,currentFrame+1);
        set(slider,'Value',currentFrame);
        updateDisplay();
```

```
    end

    function previousFrame(~,~)
        isPlaying = false;
        set(playButton,'String','PLAY');

        currentFrame = max(1,currentFrame-1);
        set(slider,'Value',currentFrame);
        updateDisplay();
    end

    function resetFrame(~,~)
        isPlaying = false;
        frameAccum = 0;
        set(playButton,'String','PLAY');

        currentFrame = 1;
        set(slider,'Value',1);
        updateDisplay();
    end

    function changeMode(src,~)
        poseMode = get(src,'Value');
        updateDisplay();
    end

    function changeSpeed(src,~)
        speedIdx = max(1,min(numel(SPEED_VALUES),round(get(src,'Value'))));
        playSpeed = SPEED_VALUES(speedIdx);
        frameAccum = 0;

        % IMPORTANT: there is intentionally NO timerObj.Period = ...
        % (that was the cause of the sub-millisecond timer warning).
    end

    function resetCamera(~,~)
        view(ax,VIEW_AZ,VIEW_EL);
        axis(ax,'equal');
        xlim(ax,AX_X);
        ylim(ax,AX_Y);
        zlim(ax,AX_Z);
    end
```

==========

==========

DISPLAY ==========

```
    function updateDisplay()
        if ~isvalidFigure(fig)
            return;
        end
```

```
        currentFrame = max(1,min(N,currentFrame));

        actual  = actualHistory(currentFrame,:);     % radians, raw
        desired = desiredHistory(currentFrame,:);
        target  = targetHistory(currentFrame,:);

        switch poseMode
            case 1
                displayAngles = target;
                poseName = 'Gesture Target';
                if ~hasLabels
                    poseName = 'Gesture Target (no labels: controller
desired)';
                end
            case 2
                displayAngles = actual;
                poseName = 'Actual PID';
            otherwise
                displayAngles = (1-BLEND_ALPHA)*actual + BLEND_ALPHA*target;
                poseName = 'Blend';
        end

        % Only the DRAWN pose is clamped; the panel shows the raw values.
        drawAngles = clampForVisualization(displayAngles);
        wasClamped = any(abs(drawAngles-displayAngles) > 1e-9);

        drawHand(ax,drawAngles);

        label        = getGestureNumber(currentFrame);
        gestureLabel = getGestureLabel(currentFrame);

        errorDeg = rad2deg(desired-actual);
        rmsError = sqrt(mean(errorDeg.^2));
        timeSec  = (currentFrame-1)*dt;

        set(frameText,'String',sprintf( ...
            'Frame: %d / %d   |   t = %.3f s',currentFrame,N,timeSec));

        infoString = sprintf([ ...
            'PROSTHETIC HAND CONTROLLER\n\n' ...
            'Frame       : %d / %d\n' ...
            'Time        : %.3f sec\n\n' ...
            'Gesture     : %s\n' ...
            'Class       : %s\n' ...
            'Pose source : %s\n\n' ...
            'RMS error   : %.2f deg  (desired - actual)'], ...
            currentFrame,N,timeSec, ...
            gestureLabel,formatLabel(label), ...
            poseName,rmsError);

        set(infoText,'String',splitlines(infoString));

        angleString = sprintf('FINGER ANGLES (deg)\n');
```

```
        angleString = sprintf('%s%-7s %8s %8s %8s\n',angleString, ...
            ' ','Pose','Actual','Target');

        for i = 1:5
            angleString = sprintf('%s%-7s %8.1f %8.1f %8.1f\n', ...
                angleString,FINGER_NAMES{i}, ...
                rad2deg(displayAngles(i)), ...
                rad2deg(actual(i)), ...
                rad2deg(target(i)));
        end

        if wasClamped
            angleString = sprintf(['%s\nPose outside the drawable range ' ...
                '(thumb 0-80, fingers 0-95 deg)\nis drawn clamped.'], ...
                angleString);
        end

        set(angleText,'String',splitlines(angleString));

        title(ax,sprintf('3-D PROSTHETIC HAND - %s',gestureLabel), ...
            'Color','w','FontSize',16,'FontWeight','bold');

        drawnow limitrate;
    end

    function printFrameAudit()
        % Prints values straight from the loaded arrays so you can compare
        % them with what the GUI shows for the same frame.
        auditFrames = unique(min(N,max(1,[1 2 3 101 102 N])));
        fprintf('FRAME AUDIT (raw loaded data, degrees)\n');
        fprintf('%7s %6s | %-27s | %-27s\n','frame','label', ...
                'target T  I  M  R  L','actual T  I  M  R  L');
        for fa = auditFrames
            fprintf('%7d %6s | %s | %s\n',fa, ...
                formatLabel(gestureVector(fa)), ...
                sprintf('%5.1f',rad2deg(targetHistory(fa,:))), ...
                sprintf('%5.1f',rad2deg(actualHistory(fa,:))));
        end
        fprintf('\n');
    end
```

==========

==========

GESTURE INFORMATION

==========

```
    function label = getGestureNumber(frame)
        label = NaN;

        if isempty(gestureVector) || frame > numel(gestureVector)
            return;
```

```
        end

        value = gestureVector(frame);

        if isnumeric(value) && isscalar(value) && isfinite(value)
            label = double(value);
        end
    end

    function name = getGestureLabel(frame)
        names = gestureNamesDB1();
        label = getGestureNumber(frame);

        if isfinite(label)
            idx = round(label)-LABEL_BASE+1;

            if idx >= 1 && idx <= numel(names)
                name = names{idx};
            else
                name = sprintf('Gesture %g',label);
            end
        else
            name = 'Unknown';
        end
    end
```

==========

==========

3-D HAND ==========

```
    function drawHand(parent,angles)
        % angles = [Thumb Index Middle Ring Little] in radians (clamped).
        % Palm + wrist are created ONCE and never move. Only the finger
        % group is deleted and rebuilt for every frame.
        ensureStaticHand(parent);

        if ~isempty(dynGroup) && isgraphics(dynGroup)
            delete(dynGroup);
        end

        dynGroup = hggroup('Parent',parent);

        drawThumb(dynGroup,angles(1));

        for f = 2:5
            drawFinger(dynGroup,f-1,angles(f));    % geometry index 1..4
        end
    end

    function ensureStaticHand(parent)
        if ~isempty(staticGroup) && isgraphics(staticGroup)
```

```
            return;
        end

        delete(findall(parent,'Type','light'));
        staticGroup = hggroup('Parent',parent);

        % ---- palm: rounded, tapered slab (wider than thick) ---------
        [Xp,Yp,Zp] = palmMesh();
        surface(Xp,Yp,Zp,'Parent',staticGroup, ...
            'FaceColor',COL_PALM,SURF_OPTS{:});

        % ---- thenar pad: blends the thumb base into the palm side ---
        [Xt,Yt,Zt] = ellipsoidMesh([3.4 -0.5 -1.8],[1.7 1.2 2.7],24);
        surface(Xt,Yt,Zt,'Parent',staticGroup, ...
            'FaceColor',COL_PALM,SURF_OPTS{:});

        % ---- wrist: elliptical tube joined to the palm bottom -------
        zw = linspace(-11,-3.5,12)';
        tw = linspace(0,2*pi,32);
        rx = 2.6 + 0.25*(zw+11)/7.5;
        ry = 0.52*rx;
        surface(rx.*cos(tw),ry.*sin(tw),repmat(zw,1,numel(tw)), ...
            'Parent',staticGroup,'FaceColor',COL_WRIST,SURF_OPTS{:});

        % ---- wrist end cap + dark cuff ring (socket look) -----------
        rr = linspace(0,1,2)';
        surface(rr*rx(1).*cos(tw),rr*ry(1).*sin(tw),-11*ones(2,numel(tw)),
...
            'Parent',staticGroup,'FaceColor',COL_CUFF,SURF_OPTS{:});

        zc = linspace(-11,-9.3,3)';
        surface((rx(1)+0.3).*cos(tw).*ones(3,1), ...
                (ry(1)+0.3).*sin(tw).*ones(3,1), ...
                repmat(zc,1,numel(tw)), ...
            'Parent',staticGroup,'FaceColor',COL_CUFF,SURF_OPTS{:});

        % ---- lights (created once; they survive per-frame redraws) --
        camlight(parent,'headlight');
        camlight(parent,'right');
    end

    function [X,Y,Z] = palmMesh()
        % Super-ellipsoid slab: flat-ish faces, squarish outline.
        [phi,theta] = meshgrid(linspace(-pi/2,pi/2,25), ...
                               linspace(-pi,pi,61));
        cf = sgnPow(cos(phi),0.60);
        sf = sgnPow(sin(phi),0.60);
        ct = sgnPow(cos(theta),0.55);
        st = sgnPow(sin(theta),0.55);

        Z = PALM_HALF_L*cf.*st;
        Y = PALM_HALF_T*sf;
        taper = 0.80 + 0.20*(Z+PALM_HALF_L)/(2*PALM_HALF_L);  % narrower at
wrist
```

```
    X = PALM_HALF_W*cf.*ct.*taper;
end

function [X,Y,Z] = ellipsoidMesh(c,r,n)
    [ex,ey,ez] = sphere(n);
    X = c(1)+r(1)*ex;
    Y = c(2)+r(2)*ey;
    Z = c(3)+r(3)*ez;
end

function y = sgnPow(u,expo)
    y = sign(u).*abs(u).^expo;
end

function drawFinger(grp,g,fingerAngle)
    % g = 1 Index, 2 Middle, 3 Ring, 4 Little (geometry index).
    % Three phalanges with CUMULATIVE flexion toward the palm (-Y):
    %   proximal  direction uses q1 = MCP
    %   middle    direction uses q2 = MCP + PIP
    %   distal    direction uses q3 = MCP + PIP + DIP
    theta = max(0,min(deg2rad(95),fingerAngle));

    jointAngles = min(JOINT_GAIN_FINGER.*theta, ...
                      deg2rad(JOINT_MAX_FINGER_DEG));
    q = cumsum(jointAngles);

    yaw    = deg2rad(FINGER_FAN_DEG(g));
    segLen = FINGER_LEN(g)*FINGER_SEG_FRAC;
    wf     = FINGER_WIDTH(g);

    pts = zeros(4,3);
    pts(1,:) = [FINGER_BASE_X(g) 0 FINGER_BASE_Z(g)];

    for seg = 1:3
        pts(seg+1,:) = pts(seg,:) + ...
            segLen(seg)*fingerDirection(q(seg),yaw);
    end

    drawBone(grp,pts(1,:),pts(2,:),0.82*wf,0.74*wf,COL_BONE);
    drawBone(grp,pts(2,:),pts(3,:),0.74*wf,0.66*wf,COL_BONE);
    drawBone(grp,pts(3,:),pts(4,:),0.66*wf,0.52*wf,COL_BONE);

    jointR = [0.88 0.78 0.70 0.55]*wf;   % MCP, PIP, DIP, fingertip
    for jj = 1:4
        drawJoint(grp,pts(jj,:),jointR(jj));
    end
end

function v = fingerDirection(q,yaw)
    % q   : cumulative flexion (0 = straight along +Z, toward -Y)
    % yaw : outward fan about the Y axis (+ toward the thumb)
    v = [sin(yaw)*cos(q), -sin(q), cos(yaw)*cos(q)];
end
```

```
function drawThumb(grp,thumbAngle)
    % Thumb has its own kinematics: it starts at the radial (+X) side
    % of the lower palm, points up/outward, and flexes toward the
    % palm face (-Y) and across the palm (-X) about one hinge axis.
    theta = max(0,min(deg2rad(80),thumbAngle));

    jointAngles = min(JOINT_GAIN_THUMB.*theta, ...
                      deg2rad(JOINT_MAX_THUMB_DEG));   % CMC, MCP, IP
    q   = cumsum(jointAngles);
    phi = deg2rad(THUMB_REST_DEG);
    hinge = thumbHingeAxis();

    pts = zeros(4,3);
    pts(1,:) = THUMB_BASE;

    for seg = 1:3
        restDir = [sin(phi(seg)) 0 cos(phi(seg))];
        pts(seg+1,:) = pts(seg,:) + ...
            THUMB_LEN(seg)*rotateAboutAxis(restDir,hinge,q(seg));
    end

    drawBone(grp,pts(1,:),pts(2,:),1.00,0.90,COL_BONE);   % metacarpal
    drawBone(grp,pts(2,:),pts(3,:),0.90,0.80,COL_BONE);   % proximal
    drawBone(grp,pts(3,:),pts(4,:),0.80,0.62,COL_BONE);   % distal

    jointR = [1.00 0.95 0.85 0.66];   % CMC, MCP, IP, thumb tip
    for jj = 1:4
        drawJoint(grp,pts(jj,:),jointR(jj));
    end
end

function hinge = thumbHingeAxis()
    phi1 = deg2rad(THUMB_REST_DEG(1));
    u1 = [sin(phi1) 0 cos(phi1)];
    % bend direction: mostly toward the palm face (-Y), partly -X
    nb = 0.78*[0 -1 0] + 0.62*[-cos(phi1) 0 sin(phi1)];
    nb = nb/norm(nb);
    hinge = cross(u1,nb);
    hinge = hinge/norm(hinge);
end

function w = rotateAboutAxis(v,axisUnit,q)
    % Rodrigues rotation of row vector v about unit axis by angle q.
    w = v*cos(q) + cross(axisUnit,v)*sin(q) ...
        + axisUnit*dot(axisUnit,v)*(1-cos(q));
end

function drawBone(grp,p1,p2,r1,r2,faceColor)
    v = p2-p1;
    boneLen = norm(v);

    if boneLen < 1e-9
        return;
    end
```

```
        [Xc,Yc,Zc] = cylinder([r1 r2],18);      % tapered
        Zc = Zc*boneLen;

        direction = v/boneLen;
        zAxis = [0 0 1];

        cx = cross(zAxis,direction);
        dz = max(-1,min(1,dot(zAxis,direction)));

        if norm(cx) < 1e-9
            if dz >= 0
                Rm = eye(3);
            else
                Rm = diag([1 -1 -1]);
            end
        else
            cx = cx/norm(cx);
            rotAngle = acos(dz);

            Kx = [ ...
                 0     -cx(3)  cx(2); ...
                 cx(3)   0    -cx(1); ...
                -cx(2)  cx(1)  0];

            Rm = eye(3) + sin(rotAngle)*Kx + (1-cos(rotAngle))*(Kx*Kx);
        end

        rotated = Rm*[Xc(:)';Yc(:)';Zc(:)'];

        X = reshape(rotated(1,:),size(Xc))+p1(1);
        Y = reshape(rotated(2,:),size(Yc))+p1(2);
        Z = reshape(rotated(3,:),size(Zc))+p1(3);

        surface(X,Y,Z,'Parent',grp,'FaceColor',faceColor,SURF_OPTS{:});
    end

    function drawJoint(grp,p,r)
        [X,Y,Z] = sphere(14);

        surface(r*X+p(1),r*Y+p(2),r*Z+p(3),'Parent',grp, ...
            'FaceColor',COL_JOINT,SURF_OPTS{:});
    end
```

==========

==========

DATA HELPERS ==========

```
    function filePath = locateFile(fileName,subFolder)
        % Looks next to this file, then in the current folder; with an
        % optional sub-folder first, then the folder itself.
```

```
    filePath = '';
    roots = unique({currentFolder,pwd},'stable');

    for ri = 1:numel(roots)
        candidates = {fullfile(roots{ri},fileName)};
        if ~isempty(subFolder)
            candidates = {fullfile(roots{ri},subFolder,fileName), ...
                          fullfile(roots{ri},fileName)};
        end

        for ci = 1:numel(candidates)
            if isfile(candidates{ci})
                filePath = candidates{ci};
                return;
            end
        end
    end
end

function history = prepareAngleHistory(history)
    if isempty(history) || ~isnumeric(history)
        history = [];
        return;
    end

    history = double(history);

    if size(history,2) == 5
        return;
    end

    if size(history,1) == 5
        history = history';
        return;
    end

    history = [];
end

function out = resizeHistory(in,Nout)
    out = zeros(Nout,5);

    if isempty(in)
        return;
    end

    n = min(Nout,size(in,1));
    out(1:n,:) = in(1:n,:);

    if n < Nout
        out(n+1:Nout,:) = repmat(in(n,:),Nout-n,1);
    end
end
```

```
function out = resizeVector(in,Nout)
    out = nan(Nout,1);

    if isempty(in)
        return;
    end

    in = in(:);
    n = min(Nout,numel(in));

    if n > 0
        out(1:n) = in(1:n);

        if n < Nout
            out(n+1:Nout) = in(n);
        end
    end
end

function history = convertToRadiansIfNeeded(history)
    if isempty(history)
        return;
    end

    finiteValues = abs(history(isfinite(history)));

    if isempty(finiteValues)
        history(:) = 0;
        return;
    end

    % If values look like degrees, convert to radians.
    if max(finiteValues) > (2*pi+0.25)
        history = deg2rad(history);
    end
end

function angles = clampForVisualization(angles)
    angles = double(angles);
    angles(~isfinite(angles)) = 0;

    % Physical visualization range.
    angles(1)   = min(max(angles(1),0),deg2rad(80));
    angles(2:5) = min(max(angles(2:5),0),deg2rad(95));
end

function v = prepareGestureVector(g)
    v = [];

    if isempty(g)
        return;
    end

    if isnumeric(g) || islogical(g)
```

```
        v = double(g(:));
    elseif iscell(g)
        v = nan(numel(g),1);

        for ii = 1:numel(g)
            value = g{ii};

            if isnumeric(value) && isscalar(value)
                v(ii) = double(value);
            elseif isstring(value) || ischar(value)
                parsed = str2double(string(value));

                if isfinite(parsed)
                    v(ii) = parsed;
                end
            end
        end
    end
end

function result = getVariableRecursive(root,names)
    result = [];

    if ~isstruct(root)
        return;
    end

    fields = fieldnames(root);

    % First search current level.
    for ii = 1:numel(names)
        for jj = 1:numel(fields)
            if strcmpi(fields{jj},names{ii})
                value = root.(fields{jj});

                if (isnumeric(value) || islogical(value) || ...
                        iscell(value)) && ~isempty(value)
                    result = value;
                    return;
                end
            end
        end
    end

    % Then recursively search structures.
    for jj = 1:numel(fields)
        value = root.(fields{jj});

        if isstruct(value)
            result = getVariableRecursive(value,names);

            if ~isempty(result)
                return;
            end
```

```
        end
    end
end

function tf = isvalidFigure(h)
    tf = ~isempty(h) && isscalar(h) && isgraphics(h,'figure');
end

function txt = formatLabel(label)
    if isfinite(label)
        txt = sprintf('%g',label);
    else
        txt = 'N/A';
    end
end

function out = ternary(cond,a,b)
    if cond
        out = a;
    else
        out = b;
    end
end
```

==========

==========

DB1 GESTURE -> HAND POSE

==========

```
function pose = gestureToPose(label,labelBase)
    % [Thumb Index Middle Ring Little], degrees.
    %
    % These are visualization target angles.
    % They provide a visible prosthetic-hand pose for each DB1 class.

    poseDeg = [0 0 0 0 0];

    if ~isfinite(label)
        pose = deg2rad(poseDeg);
        return;
    end

    idx = round(label)-labelBase;

    switch idx
        case 0, poseDeg = [0 0 0 0 0];  % Rest
        case 1, poseDeg = [0 65 0 0 0];  % Index flexion
        case 2, poseDeg = [0 5 0 0 0];  % Index extension
        case 3, poseDeg = [0 0 65 0 0];  % Middle flexion
        case 4, poseDeg = [0 0 5 0 0];  % Middle extension
        case 5, poseDeg = [0 0 0 65 0];  % Ring flexion
```

```
        case 6, poseDeg = [0 0 0 5 0];  % Ring extension
        case 7, poseDeg = [0 0 0 0 65];  % Little flexion
        case 8, poseDeg = [0 0 0 0 5];  % Little extension
        case 9, poseDeg = [35 0 0 0 0];  % Thumb adduction
        case 10, poseDeg = [0 0 0 0 0];  % Thumb abduction
        case 11, poseDeg = [65 0 0 0 0];  % Thumb flexion
        case 12, poseDeg = [5 0 0 0 0];  % Thumb extension
        case 13, poseDeg = [15 0 0 0 0];  % Thumb up
        case 14, poseDeg = [0 5 5 0 0];  % Index + middle
        case 15, poseDeg = [0 0 0 65 65];  % Ring + little
        case 16, poseDeg = [65 0 0 0 65];  % Thumb + little
        case 17, poseDeg = [0 0 0 0 0];  % All fingers open
        case 18, poseDeg = [65 70 70 70 70];  % Fist
        case 19, poseDeg = [45 5 70 70 70];  % Pointing index
        case 20, poseDeg = [0 5 5 5 5];  % Extended adduction
        case 21, poseDeg = [0 0 0 0 0];  % Wrist supination
        case 22, poseDeg = [0 0 0 0 0];  % Wrist pronation
        case 23, poseDeg = [0 0 0 0 0];  % Wrist axis
        case 24, poseDeg = [0 0 0 0 0];  % Wrist axis
        case 25, poseDeg = [20 20 20 20 20];  % Wrist flexion
        case 26, poseDeg = [0 0 0 0 0];  % Wrist extension
        case 27, poseDeg = [0 0 0 0 0];  % Radial deviation
        case 28, poseDeg = [0 0 0 0 0];  % Ulnar deviation
        case 29, poseDeg = [60 65 65 65 65];  % Closed hand
        case 30, poseDeg = [45 50 50 50 50];  % Large diameter grasp
        case 31, poseDeg = [55 60 60 60 60];  % Small diameter grasp
        case 32, poseDeg = [65 70 70 0 70];  % Hook grasp
        case 33, poseDeg = [35 5 60 60 60];  % Index extension grasp
        case 34, poseDeg = [55 60 60 60 60];  % Medium wrap
        case 35, poseDeg = [45 35 35 70 35];  % Ring grasp
        case 36, poseDeg = [45 60 60 60 60];  % Four finger grasp
        case 37, poseDeg = [50 55 55 55 55];  % Stick grasp
        case 38, poseDeg = [50 65 65 15 15];  % Writing tripod
        case 39, poseDeg = [55 65 65 65 65];  % Power sphere
        case 40, poseDeg = [50 60 60 15 15];  % Three finger sphere
        case 41, poseDeg = [45 55 55 10 10];  % Precision sphere
        case 42, poseDeg = [50 65 65 10 10];  % Tripod
        case 43, poseDeg = [55 50 50 5 5];  % Prismatic pinch
        case 44, poseDeg = [65 35 35 5 5];  % Tip pinch
        case 45, poseDeg = [55 55 55 55 5];  % Quadpod
        case 46, poseDeg = [60 35 35 35 5];  % Lateral grasp
        case 47, poseDeg = [0 5 5 5 5];  % Parallel extension
        case 48, poseDeg = [15 10 10 10 10];  % Extension type
        case 49, poseDeg = [50 60 60 60 60];  % Power disk
        case 50, poseDeg = [40 45 45 45 45];  % Open bottle
        case 51, poseDeg = [35 40 40 40 40];  % Turn screw
        case 52, poseDeg = [50 60 60 60 60];  % Cut with knife
        otherwise
            poseDeg = [0 0 0 0 0];
    end

    pose = deg2rad(poseDeg);
end
```

DB1 GESTURE NAMES

```
function names = gestureNamesDB1()
    names = { ...
        'Rest'
        'Index Flexion'
        'Index Extension'
        'Middle Flexion'
        'Middle Extension'
        'Ring Flexion'
        'Ring Extension'
        'Little Finger Flexion'
        'Little Finger Extension'
        'Thumb Adduction'
        'Thumb Abduction'
        'Thumb Flexion'
        'Thumb Extension'
        'Thumb Up'
        'Index + Middle Extension'
        'Ring + Little Flexion'
        'Thumb Opposing Little Finger'
        'All Fingers Abduction'
        'Fist'
        'Pointing Index'
        'Extended Fingers Adduction'
        'Wrist Supination'
        'Wrist Pronation'
        'Wrist Supination - Little Finger Axis'
        'Wrist Pronation - Little Finger Axis'
        'Wrist Flexion'
        'Wrist Extension'
        'Wrist Radial Deviation'
        'Wrist Ulnar Deviation'
        'Wrist Extension Closed Hand'
        'Large Diameter Grasp'
        'Small Diameter Grasp'
        'Fixed Hook Grasp'
        'Index Finger Extension Grasp'
        'Medium Wrap Grasp'
        'Ring Grasp'
        'Prismatic Four Finger Grasp'
        'Stick Grasp'
        'Writing Tripod Grasp'
        'Power Sphere Grasp'
        'Three Finger Sphere Grasp'
        'Precision Sphere Grasp'
        'Tripod Grasp'
        'Prismatic Pinch Grasp'
```

```
            'Tip Pinch Grasp'
            'Quadpod Grasp'
            'Lateral Grasp'
            'Parallel Extension Grasp'
            'Extension Type Grasp'
            'Power Disk Grasp'
            'Open Bottle'
            'Turn Screw'
            'Cut with Knife'};
    end
```

==========

==========

CLOSE / CLEANUP ==========

```
    function cleanupTimer(~,~)
        % Idempotent: safe from CloseRequestFcn, DeleteFcn and the timer.
        isPlaying = false;

        if ~isempty(timerObj) && isvalid(timerObj)
            if strcmp(timerObj.Running,'on')
                stop(timerObj);
            end
            delete(timerObj);
        end

        timerObj = [];
    end

    function closeFigure(~,~)
        cleanupTimer();

        if isvalidFigure(fig)
            delete(fig);
        end
    end

end
```

Published with MATLAB® R2025b